

Unidad Didáctica

**Scripting SQL, uso de variables, SQL
dinámico, Unidad III**



Introducción	4
Desarrollo de Contenidos	5
Variables	5
Declaración de Variables	5
Alcance de las Variables	6
Asignación de valores a Variables	8
CREATE	9
Database.....	9
Table	10
Type	11
USE.....	13
ALTER	14
Table	14
Column	15
DROP	17
Database.....	17
Table	18
Column	19
User.....	20
TRUNCATE	21
IDENTITY y GUIDs.....	22
Ciclos repetitivos.....	24

Estructuras de Control	26
STORED PROCEDURE.....	27
VIEW	29
Funciones Personalizadas de Usuario.....	30
Tipos de parámetros	32
Parámetros de Entrada	32
Parámetros de Salida	35
Parámetros tipo Tabla	36
Indices	39
Triggers.....	41
Cursore	46
Bibliografía y Webgrafía	53
Glosario	54

Introducción

En el vasto universo de la informática y la programación, un conjunto de términos y conceptos emerge como los pilares fundamentales en el diseño y manejo de bases de datos. En este emocionante viaje académico, nos adentraremos en el fascinante mundo de las operaciones de manipulación de datos en SQL, explorando temas cruciales como variables, CREATE, USE, ALTER, DROP y TRUNCATE. Estas herramientas nos permitirán moldear y esculpir la estructura de nuestras bases de datos para adaptarlas a las necesidades específicas de cada proyecto. Además, nos sumergiremos en el intrigante terreno de las IDENTITY y GUIDs, fundamentales para el manejo eficiente de registros únicos y la garantía de integridad en nuestras bases de datos.

Continuando con nuestra travesía académica, exploraremos los conceptos de ciclos repetitivos y estructuras de control en SQL. Estas valiosas técnicas nos dotarán del poder de automatizar tareas, facilitando el manejo y procesamiento masivo de datos. Descubriremos cómo STORED PROCEDURE y VIEW son esenciales para modularizar y simplificar la gestión de información en la base de datos, permitiéndonos reutilizar lógica y optimizar el rendimiento del sistema. Asimismo, nos aventuraremos en el mundo de las Funciones Personalizadas de Usuario, donde el ingenio y la creatividad se combinan para crear herramientas altamente especializadas que amplían las capacidades de SQL y se adaptan a nuestras necesidades específicas.

Finalmente, nuestra travesía nos llevará a explorar los misteriosos conceptos de tipos de parámetros, índices, triggers y cursores. Estos elementos poseen el poder de transformar por completo la eficiencia y la velocidad de nuestras consultas y operaciones, convirtiendo un simple sistema de almacenamiento en una máquina poderosa capaz de enfrentar desafíos de datos a gran escala.

Variables

Declaración de Variables

La declaración de variables en Transact-SQL te permite almacenar temporalmente valores que pueden ser utilizados en consultas, procedimientos almacenados u otros bloques de código. Puedes usar variables para simplificar el proceso de escritura de consultas y para hacer que tu código sea más legible y mantenible. Aquí tienes cómo declarar variables en el contexto de la base de datos "verdulería":

Supongamos que deseas declarar una variable para almacenar el nombre de un vendedor y luego utilizar esa variable para realizar una consulta. Aquí tienes un ejemplo:

```
DECLARE @nombreVendedor VARCHAR(100); -- Declarar una variable de tipo
VARCHAR

SET @nombreVendedor = 'Juan Pérez'; -- Asignar un valor a la variable

-- Utilizar la variable en una consulta

SELECT * FROM Vendedores WHERE nombre = @nombreVendedor;
```

En este ejemplo:

- Hemos declarado una variable llamada @nombreVendedor de tipo VARCHAR con una longitud máxima de 100 caracteres.
- Luego, hemos asignado el valor 'Juan Pérez' a la variable utilizando la sentencia SET.
- Finalmente, hemos utilizado la variable en una consulta para seleccionar todos los vendedores cuyo nombre coincida con el valor de la variable.

Recuerda que las variables en Transact-SQL son locales al bloque de código en el que se declaran y no están disponibles fuera de ese bloque. Puedes utilizar variables para almacenar valores de diferentes tipos de datos, como números, fechas, cadenas, etc.

Alcance de las Variables

El alcance de las variables en Transact-SQL **se refiere a la parte del código en la que una variable puede ser vista y utilizada. Las variables pueden tener diferentes niveles de alcance según dónde se declaren.** En Transact-SQL, hay dos tipos principales de alcance de variables: local y global.

Alcance Local: Las variables declaradas dentro de un bloque de código específico, como una función, un procedimiento almacenado o un lote de instrucciones SQL, tienen un alcance local. Esto significa que solo pueden ser referenciadas y utilizadas dentro de ese bloque. No son visibles ni accesibles fuera de él.

Ejemplo de alcance local:

```
CREATE PROCEDURE EjemploProcedimiento  
  
AS  
  
BEGIN  
  
    DECLARE @variableLocal INT;  
  
    SET @variableLocal = 10;  
  
    -- Otras operaciones con @variableLocal  
  
END;
```

En este caso, @variableLocal solo es visible dentro del procedimiento almacenado EjemploProcedimiento.

Alcance Global (o más amplio): Algunas variables, como las declaradas en la parte superior de un lote de instrucciones SQL (fuera de cualquier bloque), pueden tener un alcance más amplio y ser visibles en varias partes del lote. Estas variables se consideran globales en el contexto del lote actual.

Ejemplo de alcance global en un lote de instrucciones:

```
DECLARE @variableGlobal INT;
```

```
-- Bloque 1
```

```
SET @variableGlobal = 5;
```

```
-- Bloque 2
```

```
PRINT @variableGlobal;
```

Aquí, @variableGlobal se declara en el ámbito global del lote y puede ser utilizada en cualquier parte del lote.

Recuerda que el alcance global en Transact-SQL se refiere al alcance dentro del mismo lote de instrucciones, no al alcance en toda la base de datos. Las variables locales son generalmente más seguras y recomendables ya que limitan el acceso a la variable solo al bloque donde se definen.

Asignación de valores a Variables

La asignación de valores a variables en Transact-SQL **se realiza mediante la sentencia SET o mediante la asignación directa en el momento de la declaración**. Aquí tienes ejemplos de ambas formas:

Asignación mediante SET: Supongamos que tienes una variable llamada @nombreVendedor y deseas asignarle el valor 'María Rodríguez':

```
DECLARE @nombreVendedor VARCHAR(100); -- Declaración de variable
```

```
SET @nombreVendedor = 'María Rodríguez'; -- Asignación mediante SET
```

Asignación directa en la declaración: En el momento en que declaras una variable, también puedes asignarle un valor. Esto se hace utilizando la palabra clave =, justo después del tipo de dato y antes del punto y coma.

```
DECLARE @edad INT = 30; -- Declaración y asignación en una sola línea
```

Asignación mediante consulta: También puedes asignar el resultado de una consulta a una variable. Asegúrate de que la consulta devuelva solo un valor y que el tipo de dato coincida con el tipo de dato de la variable.

```
DECLARE @totalVentas DECIMAL(10, 2);
```

```
SELECT @totalVentas = SUM(kilos) FROM Ventas WHERE cod_vendedor = 1;
```


Recuerda que el tipo de dato de la variable debe coincidir con el tipo de dato del valor que estás asignando. Si intentas asignar un valor de un tipo incompatible, puede producirse un error.

La asignación de valores a variables te permite almacenar y manipular datos en tus consultas y procedimientos almacenados de manera conveniente y eficiente.

CREATE

Database

Para crear una base de datos en Transact-SQL, puedes utilizar el comando CREATE DATABASE. Aquí tienes un ejemplo de cómo crear una base de datos llamada "Verduleria":

```
CREATE DATABASE Verduleria;
```

Este comando creará una nueva base de datos llamada "Verduleria" en el servidor de bases de datos. Ten en cuenta que para ejecutar este comando, generalmente debes tener los privilegios adecuados en el servidor. Una vez creada la base de datos, puedes utilizarla para crear tablas, almacenar datos y realizar otras operaciones relacionadas con la base de datos.

Recuerda que este es solo un ejemplo básico de cómo crear una base de datos. En un entorno de producción real, es posible que desees especificar más opciones, como la ubicación de los archivos de datos y registros, el tamaño inicial, etc. La sintaxis exacta y las opciones disponibles pueden variar según el sistema de gestión de bases de datos que estés utilizando.

Table

Aquí tienes un ejemplo de cómo **crear una tabla en la base de datos** "Verduleria":

```
-- Usar la base de datos
```

```
USE Verduleria;
```

```
-- Crear tabla Ejemplo
```

```
CREATE TABLE Ejemplo (
```

```
    id INT PRIMARY KEY,
```

```
    nombre VARCHAR(100),
```

```
    edad INT
```

```
);
```

En este ejemplo, he creado una tabla llamada "Ejemplo" en la base de datos "Verduleria". La tabla tiene tres columnas: "id", "nombre" y "edad". La columna "id" se define como clave primaria (PRIMARY KEY) para asegurarse de que cada registro tenga un valor único en esta columna.

Puedes personalizar esta estructura de tabla según tus necesidades. Simplemente ajusta los nombres de las columnas y los tipos de datos según lo que desees almacenar en la tabla.

Type

En Transact-SQL, el comando CREATE TYPE **se utiliza para crear tipos de datos definidos por el usuario**, también conocidos como tipos de datos personalizados. Estos tipos de datos personalizados te permiten crear estructuras de datos más complejas y reutilizables que pueden contener varios valores. Aquí tienes un ejemplo de cómo crear un tipo de datos personalizado llamado "Direccion":

-- Crear el tipo de datos personalizado

```
CREATE TYPE Direccion AS TABLE (
```

```
    calle VARCHAR(100),
```

```
    numero INT,
```

```
    ciudad VARCHAR(50),
```

```
    codigo_postal VARCHAR(10)
```

```
);
```

En este ejemplo, se ha creado un tipo de datos personalizado llamado "Direccion" que consiste en una estructura de tabla con las columnas calle, numero, ciudad y codigo_postal. Luego, puedes utilizar este tipo de datos personalizado en la definición de tablas o como tipo de parámetro en procedimientos almacenados. Aquí hay un ejemplo de cómo podrías usar el tipo de datos "Direccion" en una tabla:

-- Crear una tabla que utiliza el tipo de datos personalizado

```
CREATE TABLE Clientes (
```

```
    id INT PRIMARY KEY,
```

```
    nombre VARCHAR(100),
```

```
    direccion Direccion
```

```
);
```

En este ejemplo, la tabla "Clientes" tiene una columna llamada "direccion" que utiliza el tipo de datos personalizado "Direccion".

Los tipos de datos personalizados son útiles cuando necesitas definir estructuras de datos más complejas que se repiten en varias tablas o procedimientos almacenados. Puedes crear tus propios tipos de datos para mejorar la claridad y reutilización del código. Ten en cuenta que la capacidad de crear tipos de datos personalizados puede variar según el sistema de gestión de bases de datos específico que estés utilizando. Los ejemplos anteriores son compatibles con SQL Server, pero la sintaxis podría ser diferente en otros sistemas de bases de datos.

Para agregar datos a la tabla "Clientes" que has definido en el ejemplo anterior, puedes utilizar la sentencia INSERT INTO. Aquí tienes un ejemplo de cómo agregar un nuevo cliente con su información de nombre y dirección:

```
-- Insertar datos en la tabla Clientes
```

```
INSERT INTO Clientes (id, nombre, direccion)
```

```
VALUES (1, 'Juan Pérez',
```

```
    (SELECT calle, numero, ciudad, codigo_postal
```

```
    FROM Direccion
```

```
    WHERE calle = 'Calle Principal' AND numero = 123)
```

```
);
```

En este ejemplo:

- Utilizamos la sentencia INSERT INTO para agregar datos a la tabla "Clientes".
- Especificamos las columnas en las que deseamos insertar datos: id, nombre y direccion.

- Utilizamos la cláusula VALUES para proporcionar los valores que deseamos insertar en cada columna.
- Para la columna direccion, utilizamos una subconsulta para obtener los valores de dirección desde la tabla "Direccion".
- Este es solo un ejemplo de cómo agregar un cliente a la tabla. Puedes repetir la sentencia INSERT INTO para agregar más clientes según sea necesario.

Asegúrate de que los valores que estás insertando coincidan con los tipos de datos definidos en la tabla y que los datos sean válidos.

USE

La instrucción USE se utiliza en Transact-SQL para cambiar la base de datos actual que se está utilizando en una sesión de trabajo. Esto significa que todas las operaciones posteriores se realizarán en la base de datos especificada hasta que se cambie nuevamente la base de datos utilizando otra instrucción USE.

Aquí tienes un ejemplo de cómo usar la instrucción USE para cambiar a la base de datos "Verduleria" que hemos estado utilizando en ejemplos anteriores:

```
USE Verduleria;
```

Después de ejecutar esta instrucción, todas las operaciones que realices, como crear tablas, insertar datos, ejecutar consultas, etc., se llevarán a cabo en la base de datos "Verduleria" hasta que cambies de base de datos nuevamente mediante otra instrucción USE. Es importante recordar que, antes de utilizar una base de datos en Transact-SQL, debes asegurarte de haberla creado previamente utilizando la instrucción CREATE DATABASE. Si intentas cambiar a una base de datos que no existe, obtendrás un error.

ALTER

Table

La sentencia ALTER TABLE se utiliza en Transact-SQL para realizar cambios en una tabla existente. Puedes utilizar esta sentencia para agregar, modificar o eliminar columnas, restricciones u otras propiedades de una tabla. Aquí tienes ejemplos de cómo usar ALTER TABLE:

Agregar una columna:

Supongamos que quieres agregar una columna llamada "email" a la tabla "Clientes" que ya tienes:

```
ALTER TABLE Clientes
```

```
ADD email VARCHAR(100);
```

Modificar una columna:

Si deseas cambiar el tipo de datos de una columna existente, por ejemplo, cambiar la longitud de la columna "nombre" de 50 a 100 caracteres:

```
ALTER TABLE Clientes
```

```
ALTER COLUMN nombre VARCHAR(100);
```

Eliminar una columna:

Si necesitas eliminar una columna de la tabla, como la columna "direccion":

```
ALTER TABLE Clientes
```

```
DROP COLUMN direccion;
```

Agregar una restricción:

Puedes agregar restricciones a las columnas existentes. Por ejemplo, agregar una restricción NOT NULL a la columna "nombre":

```
ALTER TABLE Clientes
```

```
ALTER COLUMN nombre VARCHAR(100) NOT NULL;
```

Estos son solo algunos ejemplos de cómo puedes utilizar la sentencia ALTER TABLE para realizar cambios en una tabla existente. Antes de ejecutar cualquiera de estas instrucciones, asegúrate de tener una copia de seguridad de tus datos, especialmente si estás realizando cambios significativos en la estructura de la tabla.

Column

La instrucción ALTER COLUMN **se utiliza** en Transact-SQL junto con la sentencia ALTER TABLE **para realizar modificaciones en una columna existente de una tabla**. Puedes usar ALTER COLUMN para cambiar el tipo de datos, la longitud, agregar restricciones y realizar otras modificaciones en una columna específica. Aquí tienes ejemplos de cómo usar ALTER COLUMN:

Cambiar el tipo de datos y longitud de una columna:

Supongamos que deseas cambiar el tipo de datos y la longitud de la columna "nombre" en la tabla "Clientes" de VARCHAR(50) a VARCHAR(100):

```
ALTER TABLE Clientes
```

```
ALTER COLUMN nombre VARCHAR(100);
```

Agregar una restricción NOT NULL:

Si deseas asegurarte de que la columna "email" no pueda contener valores nulos:

```
ALTER TABLE Clientes
```

```
ALTER COLUMN email VARCHAR(100) NOT NULL;
```

Quitar la restricción NOT NULL:

Si deseas permitir valores nulos en la columna "email" después de haberla configurado como NOT NULL:

```
ALTER TABLE Clientes
```

```
ALTER COLUMN email VARCHAR(100) NULL;
```

Cambiar el valor predeterminado:

Si necesitas cambiar el valor predeterminado de una columna, por ejemplo, cambiar el valor predeterminado de la columna "edad" a 25:

```
ALTER TABLE Clientes
```

```
ALTER COLUMN edad INT DEFAULT 25;
```

Eliminar una restricción DEFAULT:

Si deseas eliminar el valor predeterminado de la columna "edad":

```
ALTER TABLE Clientes
```

```
ALTER COLUMN edad DROP DEFAULT;
```

Recuerda que realizar cambios en las columnas de una tabla puede afectar a los datos existentes y, en algunos casos, puede requerir conversiones de datos. Antes de realizar modificaciones importantes en la estructura de la tabla, es recomendable respaldar tus datos y comprender completamente el impacto de los cambios.

DROP

Database

La instrucción DROP DATABASE **se utiliza en Transact-SQL para eliminar una base de datos existente junto con todos sus objetos, como tablas, vistas, procedimientos almacenados, etc.** Esta acción es irreversible y eliminará permanentemente todos los datos y objetos contenidos en la base de datos. Por lo tanto, es importante tener mucho cuidado al usar esta instrucción.

Aquí tienes un ejemplo de cómo utilizar DROP DATABASE para eliminar la base de datos "Verduleria":

```
USE master; -- Cambiar a la base de datos master (sistema)
```

```
DROP DATABASE Verduleria; -- Eliminar la base de datos Verduleria
```

En este ejemplo:

- Utilizamos la instrucción USE para cambiar a la base de datos "master", que es la base de datos del sistema donde se almacenan los metadatos y otras configuraciones.
- Luego utilizamos DROP DATABASE para eliminar la base de datos "Verduleria".

Recuerda que al eliminar una base de datos, todos los datos almacenados en ella se perderán de forma permanente. Asegúrate de haber respaldado tus datos importantes antes de ejecutar esta instrucción.

Ten en cuenta también que el comando DROP DATABASE puede requerir ciertos permisos y autorizaciones en el sistema de gestión de bases de datos. Asegúrate de tener los permisos adecuados antes de intentar eliminar una base de datos.

Table

La instrucción DROP TABLE se utiliza en Transact-SQL para eliminar una tabla específica de una base de datos. Al igual que con la instrucción DROP DATABASE, es una acción irreversible y eliminará permanentemente la tabla y todos los datos contenidos en ella. Por lo tanto, debes ser muy cuidadoso al usar esta instrucción.

Aquí tienes un ejemplo de cómo utilizar DROP TABLE para eliminar la tabla "Clientes":

```
USE Verduleria; -- Cambiar a la base de datos donde se encuentra la tabla
```

```
DROP TABLE Clientes; -- Eliminar la tabla Clientes
```

En este ejemplo:

- Utilizamos la instrucción USE para cambiar a la base de datos "Verduleria", donde se encuentra la tabla que queremos eliminar.
- Luego utilizamos DROP TABLE seguido del nombre de la tabla ("Clientes") para eliminar la tabla.

Recuerda que al eliminar una tabla, todos los datos contenidos en ella se perderán de forma permanente. Asegúrate de haber respaldado tus datos importantes antes de ejecutar esta instrucción.

Al igual que con DROP DATABASE, el comando DROP TABLE también puede requerir ciertos permisos y autorizaciones en el sistema de gestión de bases de datos. Asegúrate de tener los permisos adecuados antes de intentar eliminar una tabla.

Column

A veces, es necesario eliminar una o más columnas obsoletas o no utilizadas en una tabla. Para hacer esto, puedes usar la instrucción ALTER TABLE y DROP COLUMN de la siguiente manera:

```
ALTER TABLE table_name
```

```
DROP COLUMN column_name;
```

En esta sintaxis:

1. Primero, debes especificar el nombre de la tabla de la que deseas eliminar la columna.
2. En segundo lugar, especifica el nombre de la columna que deseas eliminar.
3. Si la columna que deseas eliminar tiene una restricción CHECK, primero debes eliminar la restricción antes de eliminar la columna. Además, SQL Server no te permite eliminar una columna que tenga una restricción PRIMARY KEY o FOREIGN KEY.

Si deseas eliminar varias columnas a la vez, utiliza la siguiente sintaxis:

```
ALTER TABLE table_name
```

```
DROP COLUMN column_name_1, column_name_2,...;
```

En esta sintaxis, se especifican las columnas que deseas eliminar como una lista de columnas separadas por comas en la cláusula DROP COLUMN.

Supongamos que deseas eliminar la columna "estado_civil" de la tabla "Vendedores" en la base de datos "Verduleria". Aquí está cómo lo harías:

Eliminar una columna individual:

```
USE Verduleria;
```

```
-- Eliminar la columna "estado_civil" de la tabla "Vendedores"
```

```
ALTER TABLE Vendedores
```

```
DROP COLUMN estado_civil;
```

Si también deseas eliminar la columna "codigo_postal" de la tabla "Clientes", puedes hacerlo de la siguiente manera:

Eliminar varias columnas a la vez:

```
USE Verduleria;
```

```
-- Eliminar las columnas "codigo_postal" y "email" de la tabla "Clientes"
```

```
ALTER TABLE Clientes
```

```
DROP COLUMN codigo_postal, email;
```

Recuerda que estos son ejemplos hipotéticos y deberías asegurarte de que las columnas que deseas eliminar no contengan datos críticos y que comprendas el impacto en la estructura de la base de datos antes de ejecutar estas instrucciones.

User

Para eliminar un usuario en Transact-SQL, puedes utilizar la instrucción DROP USER. Sin embargo, ten en cuenta que esta acción debe realizarse con precaución, ya que eliminar un usuario puede tener implicaciones en la seguridad y los permisos de la base de datos. Aquí tienes un ejemplo de cómo usar DROP USER:

Supongamos que deseas eliminar el usuario "UsuarioEjemplo" de la base de datos "Verduleria":

```
USE Verduleria;
```

```
-- Eliminar el usuario "UsuarioEjemplo"
```

```
DROP USER UsuarioEjemplo;
```

En este ejemplo, la instrucción USE se utiliza para cambiar a la base de datos "Verduleria", donde deseas eliminar el usuario. Luego, DROP USER se utiliza para eliminar el usuario "UsuarioEjemplo".

Recuerda que al eliminar un usuario, se eliminarán sus permisos y cualquier objeto asociado a él. Asegúrate de entender el impacto de esta acción y de tener la autorización adecuada para eliminar usuarios.

TRUNCATE

La instrucción TRUNCATE TABLE se utiliza en Transact-SQL para **eliminar todos los registros (filas) de una tabla, pero mantiene la estructura de la tabla y la posibilidad de ingresar nuevos registros posteriormente**. Es una forma más eficiente de eliminar todos los datos de una tabla en comparación con la instrucción DELETE porque no genera tantos registros de registro y no registra cada fila eliminada individualmente.

Aquí tienes un ejemplo de cómo usar TRUNCATE TABLE en el contexto de la base de datos "Verduleria". Supongamos que deseas eliminar todos los registros de la tabla "Ventas":

```
USE Verduleria;
```

-- Eliminar todos los registros de la tabla "Ventas"

TRUNCATE TABLE Ventas;

En este ejemplo, la instrucción USE se utiliza para cambiar a la base de datos "Verduleria", donde deseas truncar la tabla. Luego, TRUNCATE TABLE se utiliza para eliminar todos los registros de la tabla "Ventas".

Es importante tener en cuenta que TRUNCATE TABLE no se puede usar en tablas que estén involucradas en relaciones de clave externa, tablas que participan en índices únicos, ni en tablas que tengan activada la replicación o estén siendo auditadas. En esos casos, es posible que necesites utilizar la instrucción DELETE en su lugar.

Recuerda que TRUNCATE TABLE es una operación irreversible y eliminará todos los datos de la tabla. Asegúrate de tener una copia de seguridad adecuada antes de realizar la operación.

IDENTITY y GUIDs

Tanto IDENTITY como GUID **son conceptos relacionados con la generación de valores únicos en una base de datos. Sin embargo, se utilizan en contextos diferentes y tienen características distintas.** Aquí tienes una explicación sobre ambos:

IDENTITY: Es una propiedad que se puede aplicar a una columna en una tabla de una base de datos. Cuando se define una columna como IDENTITY, la base de datos asigna automáticamente valores numéricos únicos a medida que se insertan filas en

la tabla. Por lo general, se usa para crear claves primarias únicas o para llevar un control de la secuencia de inserción.

Ejemplo de uso de IDENTITY:

```
CREATE TABLE Ejemplo (  
  
    id INT IDENTITY(1, 1) PRIMARY KEY,  
  
    nombre VARCHAR(100)  
  
);
```

En este ejemplo, la columna "id" se define como una columna IDENTITY, lo que significa que la base de datos generará automáticamente valores únicos incrementales cada vez que se inserta una nueva fila en la tabla.

GUID (Globally Unique Identifier): Un GUID es un valor único a nivel global que se genera utilizando un algoritmo específico. A diferencia de las claves generadas con IDENTITY, los GUIDs no siguen una secuencia numérica y son mucho menos propensos a colisiones (valores duplicados). Los GUIDs son especialmente útiles en sistemas distribuidos donde múltiples bases de datos pueden generar valores únicos sin la necesidad de una coordinación central.

Ejemplo de uso de GUID:

```
CREATE TABLE EjemploGuid (  
  
    id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),  
  
    nombre VARCHAR(100)  
  
);
```

En este ejemplo, la columna "id" se define como un UNIQUEIDENTIFIER y se utiliza la función NEWID() para generar un GUID único como valor predeterminado.

En resumen, IDENTITY es útil para generar valores numéricos únicos en una secuencia y se usa comúnmente para claves primarias. Los GUIDs son útiles cuando se necesita garantizar la unicidad global y no se requiere una secuencia numérica predecible. La elección entre IDENTITY y GUID depende de las necesidades específicas de tu base de datos y la arquitectura de tu sistema.

Ciclos repetitivos

En Transact-SQL, los ciclos repetitivos, también conocidos como bucles, se pueden implementar utilizando principalmente las instrucciones WHILE y BEGIN...END. Aquí te muestro cómo utilizar estas instrucciones para crear ciclos repetitivos:

WHILE Loop:

La instrucción WHILE **se utiliza para ejecutar un bloque de código mientras se cumple una condición especificada**. Aquí tienes un ejemplo de cómo usar WHILE para crear un ciclo repetitivo:

```
DECLARE @contador INT = 1;

WHILE @contador <= 10

BEGIN

    PRINT 'Iteración: ' + CAST(@contador AS VARCHAR(10));

    SET @contador = @contador + 1;

END;
```


En este ejemplo, el ciclo WHILE se ejecutará mientras @contador sea menor o igual a 10. En cada iteración, se imprimirá un mensaje que muestra la iteración actual, y el contador se incrementará en 1.

BEGIN...END Block:

Puedes combinar BEGIN...END con condiciones de control, como IF, para crear ciclos más complejos. Por ejemplo, aquí hay un ciclo que ejecuta un bloque de código tres veces:

```
DECLARE @iteraciones INT = 3;

DECLARE @contador INT = 1;

WHILE @contador <= @iteraciones

BEGIN

    PRINT 'Iteración: ' + CAST(@contador AS VARCHAR(10));

    -- Coloca aquí el código adicional que deseas ejecutar

    SET @contador = @contador + 1;

END;
```

Es importante asegurarte de que el código dentro del ciclo WHILE tenga una forma de finalizar o que la condición de control cambie para evitar un ciclo infinito. Recuerda que mientras que las instrucciones WHILE y BEGIN...END permiten crear ciclos repetitivos en Transact-SQL, también es importante considerar otras opciones y estructuras para lograr el mismo resultado, como consultas recursivas y funciones escalares. El enfoque a utilizar depende de la situación y la lógica específica que estés implementando.

Estructuras de Control

En Transact-SQL, las estructuras de control son elementos fundamentales para controlar el flujo de ejecución de un programa. **Permiten realizar acciones condicionales y repetitivas en base a ciertas condiciones.** Las estructuras de control principales en Transact-SQL son las siguientes:

IF...ELSE: Permite ejecutar un bloque de código si se cumple una condición y, opcionalmente, otro bloque si la condición no se cumple.

```
DECLARE @valor INT = 10;

IF @valor > 5

BEGIN

    PRINT 'El valor es mayor que 5';

END

ELSE

BEGIN

    PRINT 'El valor no es mayor que 5';

END;
```

CASE: Proporciona una forma de realizar evaluaciones condicionales en función de múltiples condiciones.

```
DECLARE @opcion INT = 2;

SELECT

CASE

    WHEN @opcion = 1 THEN 'Opción 1'

    WHEN @opcion = 2 THEN 'Opción 2'

    ELSE 'Opción desconocida'

END;
```

Estas son algunas de las estructuras de control básicas en Transact-SQL. Dependiendo de tus necesidades, podrías utilizar combinaciones de estas estructuras para lograr la lógica deseada en tus programas. Recuerda que una buena comprensión de las estructuras de control es esencial para desarrollar lógica de programación efectiva y eficiente.

STORED PROCEDURE

Un Stored Procedure (Procedimiento Almacenado) es un objeto de base de datos en Transact-SQL que encapsula una serie de instrucciones SQL en una unidad lógica y reutilizable. Los procedimientos almacenados **se almacenan en la base de datos y pueden ser llamados y ejecutados desde diferentes aplicaciones o scripts**, lo que permite modularizar y centralizar la lógica de negocio.

Aquí tienes un ejemplo básico de cómo crear y ejecutar un Stored Procedure en la base de datos "Verduleria":

Creación de un Stored Procedure:

```
USE Verduleria;  
  
CREATE PROCEDURE ObtenerVentasPorVendedor  
  
    @vendedorId INT  
  
AS  
  
BEGIN  
  
    SELECT *  
  
    FROM Ventas  
  
    WHERE cod_vendedor = @vendedorId;  
  
END;
```

En este ejemplo, hemos creado un Stored Procedure llamado "ObtenerVentasPorVendedor" que toma un parámetro @vendedorId y devuelve todas las ventas realizadas por ese vendedor.

Ejecución de un Stored Procedure:

Una vez creado, puedes ejecutar el procedimiento almacenado de la siguiente manera:

```
EXEC ObtenerVentasPorVendedor @vendedorId = 1;
```

Esto ejecutará el procedimiento almacenado "ObtenerVentasPorVendedor" con el valor 1 como el valor del parámetro @vendedorId.

Los Stored Procedures son útiles por varias razones, como:

- Reutilización de código: Puedes crear una lógica compleja una vez y reutilizarla en diferentes partes de tu aplicación.
- Mejora del rendimiento: Los procedimientos almacenados pueden ser compilados y almacenados en caché, lo que puede mejorar el rendimiento de la base de datos.
- Seguridad: Puedes otorgar permisos específicos de ejecución en procedimientos almacenados sin otorgar acceso directo a las tablas subyacentes.

VIEW

Una View (Vista) es una consulta SQL predefinida almacenada en la base de datos como un objeto. Una vista **permite presentar una parte específica de los datos en una forma más organizada y legible, actuando como una "ventana" virtual a los datos subyacentes de una o más tablas**. Las vistas son especialmente útiles para simplificar la complejidad de las consultas y para proporcionar una capa de abstracción sobre los datos.

Aquí tienes un ejemplo de cómo crear y utilizar una vista en la base de datos "Verduleria":

Creación de una View:

```
USE Verduleria;
```

```
CREATE VIEW VentasPorVendedor AS
```

```
SELECT v.*, ve.nombre, ve.apellido
```

```
FROM Ventas v
```

```
JOIN Vendedores ve ON v.cod_vendedor = ve.id_vendedor;
```

En este ejemplo, hemos creado una vista llamada "VentasPorVendedor" que combina datos de las tablas "Ventas" y "Vendedores" para mostrar información sobre las ventas y los vendedores.

Uso de una View:

Una vez creada, puedes consultar una vista de manera similar a una tabla:

```
SELECT * FROM VentasPorVendedor;
```

Esto ejecutará la consulta definida en la vista y mostrará los resultados como si estuvieras consultando una tabla real.

Las vistas son útiles por varias razones, como:

- Simplificación de consultas: Las vistas pueden encapsular lógica compleja, permitiendo a los usuarios ejecutar consultas más simples y legibles.
- Seguridad: Puedes limitar los datos que se muestran en una vista, proporcionando una capa de seguridad adicional al ocultar columnas o filas sensibles.
- Abstracción de datos: Las vistas pueden ocultar la estructura interna de las tablas y proporcionar una vista lógica de los datos que es más fácil de entender.

Funciones Personalizadas de Usuario

En Transact-SQL, puedes crear funciones personalizadas de usuario para encapsular lógica reutilizable y realizar cálculos o manipulaciones de datos específicos. **Las funciones permiten encapsular la lógica en un objeto que puede ser llamado desde consultas SQL, procedimientos almacenados, vistas y más.** Hay dos tipos principales de funciones: funciones escalares y funciones de tabla.

Aquí tienes un ejemplo de cómo crear y utilizar una función escalar de usuario en la base de datos "Verduleria":

Creación de una función escalar de usuario:

```
USE Verduleria;

CREATE FUNCTION CalcularTotalVenta

(

    @precio DECIMAL(10, 2),

    @cantidad DECIMAL(10, 2)

)

RETURNS DECIMAL(10, 2)

AS

BEGIN

    DECLARE @total DECIMAL(10, 2);

    SET @total = @precio * @cantidad;

    RETURN @total;

END;
```

En este ejemplo, hemos creado una función escalar de usuario llamada "CalcularTotalVenta" que toma el precio y la cantidad de un producto como parámetros y devuelve el total calculado.

Uso de una función escalar de usuario:

Puedes usar la función en una consulta SQL de esta manera:

```
SELECT nombre, precio, cantidad, dbo.CalcularTotalVenta(precio, cantidad) AS total  
FROM Productos;
```

Aquí hemos utilizado la función `CalcularTotalVenta` para calcular el total para cada producto en la tabla "Productos".

Las funciones de tabla son otro tipo de funciones que devuelven conjuntos de resultados (tablas). Estas funciones pueden ser utilizadas en las cláusulas `FROM` de las consultas, como si fueran tablas reales.

Tipos de parámetros

Parámetros de Entrada

Los parámetros de entrada **son valores que se pasan a una función, procedimiento almacenado o vista para ser utilizados en su lógica**. Estos parámetros permiten que la función o procedimiento acepte valores externos y los utilice para realizar cálculos, filtrar datos o llevar a cabo cualquier otro tipo de operación.

En Transact-SQL, puedes definir parámetros de entrada en funciones personalizadas de usuario, procedimientos almacenados y otros objetos. Aquí tienes ejemplos de cómo definir y utilizar parámetros de entrada en procedimientos almacenados y funciones:

Procedimiento Almacenado con Parámetros de Entrada:

```
USE Verduleria;

CREATE PROCEDURE ObtenerVentasPorVendedor

    @vendedorId INT

AS

BEGIN

    SELECT *

    FROM Ventas

    WHERE cod_vendedor = @vendedorId;

END;
```

En este ejemplo, el procedimiento almacenado "ObtenerVentasPorVendedor" acepta un parámetro de entrada @vendedorId, que se utiliza para filtrar las ventas del vendedor correspondiente.

Función con Parámetros de Entrada:

```
USE Verduleria;

CREATE FUNCTION CalcularTotalVenta

(

    @precio DECIMAL(10, 2),

    @cantidad DECIMAL(10, 2)

)
```

```
RETURNS DECIMAL(10, 2)
```

```
AS
```

```
BEGIN
```

```
    DECLARE @total DECIMAL(10, 2);
```

```
    SET @total = @precio * @cantidad;
```

```
    RETURN @total;
```

```
END;
```

En este ejemplo, la función escalar de usuario "CalcularTotalVenta" acepta dos parámetros de entrada @precio y @cantidad, que se utilizan para calcular el total.

Ejecución de Procedimiento Almacenado con Parámetros:

Para ejecutar un procedimiento almacenado con parámetros, puedes hacer lo siguiente:

```
EXEC ObtenerVentasPorVendedor @vendedorId = 1;
```

Uso de Función con Parámetros:

Para usar una función con parámetros en una consulta, puedes hacer lo siguiente:

```
SELECT nombre, precio, cantidad, dbo.CalcularTotalVenta(precio, cantidad) AS total
```

```
FROM Productos;
```

Los parámetros de entrada permiten que tus objetos de base de datos sean más flexibles y reutilizables, ya que pueden ser adaptados a diferentes contextos y datos.

Asegúrate de que los parámetros de entrada estén correctamente definidos y sean utilizados adecuadamente en tu lógica de programación.

Parámetros de Salida

Los parámetros de salida son valores que una función o procedimiento almacenado devuelve a la parte que llamó a ese objeto. A diferencia de los parámetros de entrada, que se pasan a un objeto para su uso interno, los parámetros de salida permiten que un objeto devuelva información específica después de su ejecución.

En Transact-SQL, puedes definir parámetros de salida en procedimientos almacenados y funciones. Aquí tienes ejemplos de cómo definir y utilizar parámetros de salida:

Procedimiento Almacenado con Parámetros de Salida:

```
USE Verduleria;
```

```
CREATE PROCEDURE ObtenerCantidadVentas
```

```
    @vendedorId INT,
```

```
    @cantidadVentas INT OUTPUT
```

```
AS
```

```
BEGIN
```

```
    SELECT @cantidadVentas = COUNT(*)
```

```
    FROM Ventas
```

```
    WHERE cod_vendedor = @vendedorId;
```

```
END;
```

En este ejemplo, el procedimiento almacenado "ObtenerCantidadVentas" acepta un parámetro de entrada @vendedorId y un parámetro de salida @cantidadVentas. El procedimiento calcula y asigna la cantidad de ventas del vendedor al parámetro de salida.

Uso de Procedimiento Almacenado con Parámetros de Salida:

```
DECLARE @ventas INT;
```

```
EXEC ObtenerCantidadVentas @vendedorId = 1, @cantidadVentas = @ventas
```

```
OUTPUT;
```

```
PRINT 'Cantidad de ventas: ' + CAST(@ventas AS VARCHAR(10));
```

En este ejemplo, declaramos una variable @ventas y luego ejecutamos el procedimiento almacenado "ObtenerCantidadVentas" pasando @vendedorId y recibiendo el valor de @cantidadVentas como parámetro de salida.

Recuerda que los parámetros de salida son una forma de comunicar información específica desde un objeto (como un procedimiento almacenado) de vuelta a la parte que lo llamó. Pueden ser útiles para devolver resultados de cálculos, contadores, identificadores generados y más.

Parámetros tipo Tabla

Los parámetros tipo tabla en Transact-SQL te permiten pasar conjuntos de datos (tablas) como parámetros a funciones o procedimientos almacenados. Esto puede ser útil cuando necesitas trabajar con varios valores en lugar de un solo valor. Los parámetros tipo tabla son especialmente útiles para minimizar la necesidad de pasar listas de valores como cadenas de texto y para realizar operaciones en conjuntos de datos más grandes y estructurados.

Aquí tienes un ejemplo de cómo definir y utilizar parámetros tipo tabla en Transact-SQL:

Definición de Tipo de Tabla:

Primero, necesitas definir un tipo de tabla personalizado que especifica la estructura de la tabla que se utilizará como parámetro. Esto se hace utilizando la instrucción CREATE TYPE.

```
USE Verduleria;

CREATE TYPE dbo.VentasTipoTabla AS TABLE

(
    cod_vendedor INT,
    cod_producto INT,
    fecha DATE,
    kilos DECIMAL(10, 2)
);
```

Procedimiento Almacenado con Parámetro Tipo Tabla:

Luego, puedes crear un procedimiento almacenado que acepte un parámetro tipo tabla y realice operaciones en los datos contenidos en ese parámetro.

```
USE Verduleria;

CREATE PROCEDURE ProcesarVentas

    @ventas dbo.VentasTipoTabla READONLY
```

AS

BEGIN

-- Hacer algo con @ventas, como insertarlos en una tabla o calcular totales

SELECT cod_vendedor, SUM(kilos) AS TotalKilos

FROM @ventas

GROUP BY cod_vendedor;

END;

Uso del Procedimiento Almacenado con Parámetro Tipo Tabla:

Puedes llamar al procedimiento almacenado y pasar una tabla como parámetro de la siguiente manera:

DECLARE @misVentas dbo.VentasTipoTabla;

INSERT INTO @misVentas (cod_vendedor, cod_producto, fecha, kilos)

VALUES (1, 101, '2023-08-01', 5.2),

(2, 102, '2023-08-02', 8.7);

EXEC ProcesarVentas @ventas = @misVentas;

En este ejemplo, hemos creado una variable tipo tabla @misVentas, insertado datos en ella y luego llamado al procedimiento almacenado ProcesarVentas pasando la variable como parámetro.

Los parámetros tipo tabla son una herramienta poderosa para trabajar con conjuntos de datos en Transact-SQL. Pueden mejorar la eficiencia y la legibilidad del código al reducir la necesidad de manipulaciones complejas de cadenas y ayudan a evitar problemas de seguridad como la inyección SQL.

Indices

Los índices en Transact-SQL son estructuras de datos que mejoran la velocidad de búsqueda y recuperación de registros en una tabla. Los índices **permiten que las consultas se ejecuten más eficientemente al reducir la cantidad de datos que deben ser examinados**. Los índices se crean en una o varias columnas de una tabla y se utilizan para acelerar la búsqueda y recuperación de registros en función de los valores almacenados en esas columnas.

Aquí tienes información básica sobre los índices en Transact-SQL:

Creación de un Índice:

Puedes crear un índice utilizando la instrucción CREATE INDEX. Aquí hay un ejemplo de cómo crear un índice en la columna nombre de la tabla Productos en la base de datos "Verduleria":

```
USE Verduleria;
```

```
CREATE INDEX idx_Productos_Nombre ON Productos (nombre);
```

En este ejemplo, hemos creado un índice llamado idx_Productos_Nombre en la columna nombre de la tabla Productos.

Uso de Índices en Consultas:

Los índices se utilizan automáticamente en consultas para mejorar la velocidad de búsqueda. Por ejemplo, si tienes el índice idx_Productos_Nombre en la columna nombre, las consultas que involucran búsquedas o filtrado en esa columna pueden ser más rápidas.

-- Ejemplo de consulta utilizando un índice

```
SELECT * FROM Productos WHERE nombre = 'Manzana';
```

Tipos de Índices:

- **Índices Clustered:** Definen el orden físico de los datos en la tabla. Una tabla puede tener solo un índice clustered, y este determina cómo se almacenan los datos en disco.
- **Índices Non-Clustered:** Creados en una columna o conjunto de columnas distintas de la columna en la que se basa el índice clustered. Estos índices no afectan el orden físico de los datos, sino que crean una estructura adicional para mejorar la búsqueda.

Consideraciones Importantes:

- Los índices mejoran la velocidad de búsqueda, pero también ocupan espacio en disco. Demasiados índices pueden ralentizar las operaciones de escritura.
- Los índices deben ser creados en columnas que se utilizan con frecuencia en consultas.
- Las operaciones de inserción, actualización y eliminación pueden ser más lentas debido a la necesidad de mantener los índices actualizados.

La creación y el mantenimiento de índices es un aspecto crítico para optimizar el rendimiento de las consultas en una base de datos. Debes considerar

cuidadosamente qué columnas necesitan índices basados en la frecuencia de consulta y el equilibrio entre la velocidad de búsqueda y el impacto en las operaciones de escritura.

Triggers

Un trigger (desencadenador) en Transact-SQL es un tipo de objeto de base de datos que permite automatizar acciones en respuesta a eventos que ocurren en una tabla, como inserciones, actualizaciones o eliminaciones de datos. Los triggers son útiles para implementar lógica de negocio compleja o asegurarse de que ciertas acciones se realicen de manera consistente en la base de datos.

Aquí tienes información básica sobre los triggers en Transact-SQL:

Creación de un Trigger:

Puedes crear un trigger utilizando la instrucción CREATE TRIGGER. Aquí hay un ejemplo de cómo crear un trigger que se activa después de insertar datos en la tabla Ventas en la base de datos "Verduleria":

```
USE Verduleria;
```

```
CREATE TRIGGER trg_AgregarVenta
```

```
ON Ventas
```

```
AFTER INSERT
```

```
AS
```

```
BEGIN
```

```
-- Lógica a ejecutar después de una inserción en la tabla Ventas
```

-- Por ejemplo, puedes actualizar otras tablas o realizar cálculos adicionales.

END;

En este ejemplo, hemos creado un trigger llamado trg_AgregarVenta que se activará después de realizar una inserción en la tabla Ventas.

Lógica del Trigger:

Dentro del cuerpo del trigger, puedes escribir la lógica que deseas ejecutar en respuesta al evento. Esta lógica puede ser una o varias instrucciones SQL que realicen cambios en otras tablas, realicen cálculos o realicen cualquier acción requerida.

Tipos de Triggers:

- AFTER INSERT: Se ejecuta después de insertar datos en la tabla.
- AFTER UPDATE: Se ejecuta después de actualizar datos en la tabla.
- AFTER DELETE: Se ejecuta después de eliminar datos de la tabla.
- INSTEAD OF: Se ejecuta en lugar de la operación original (inserción, actualización o eliminación).

Consideraciones Importantes:

- Los triggers pueden ser poderosos, pero también pueden complicar la lógica de la base de datos si se usan en exceso.
- Debes tener cuidado con los triggers para evitar bucles infinitos si la lógica del trigger realiza cambios en la misma tabla que activa el trigger.
- Los triggers pueden afectar el rendimiento de las operaciones en la base de datos, especialmente si realizan operaciones costosas.

Los triggers son una herramienta útil para implementar lógica de negocio automatizada en una base de datos. Sin embargo, debes usarlos con precaución y asegurarte de entender cómo afectan el flujo de trabajo y el rendimiento general de tu sistema.

AFTER INSERT Trigger:

Este trigger se activa después de insertar datos en la tabla Ventas y realiza una acción adicional, en este caso, imprime un mensaje

USE Verduleria;

CREATE TRIGGER trg_AfterInsert

ON Ventas

AFTER INSERT

AS

BEGIN

PRINT 'Se ha insertado una nueva venta.';

END;

AFTER UPDATE Trigger:

Este trigger se activa después de actualizar datos en la tabla Productos y ajusta automáticamente el precio a un valor mínimo de 1 si el precio es menor a 1.

USE Verduleria;

CREATE TRIGGER trg_AfterUpdate

ON Productos

AFTER UPDATE

```
AS
```

```
BEGIN
```

```
    UPDATE p
```

```
    SET precio = 1
```

```
    FROM Productos p
```

```
    INNER JOIN inserted i ON p.id_producto = i.id_producto
```

```
    WHERE i.precio < 1;
```

```
END;
```

AFTER DELETE Trigger:

Este trigger se activa después de eliminar datos de la tabla Clientes y realiza una acción adicional, en este caso, imprime un mensaje.

```
USE Verduleria;
```

```
CREATE TRIGGER trg_AfterDelete
```

```
ON Clientes
```

```
AFTER DELETE
```

```
AS
```

```
BEGIN
```

```
    PRINT 'Se ha eliminado un cliente.';
```

```
END;
```

INSTEAD OF Trigger:

Este trigger se activa en lugar de realizar la operación original de inserción en la tabla Ventas. En este caso, se realiza una inserción en una tabla de registro antes de realizar la inserción en la tabla Ventas.

```
USE Verduleria;

CREATE TRIGGER trg_InsteadOfInsert

ON Ventas

INSTEAD OF INSERT

AS

BEGIN

    INSERT INTO Ventas_Log (cod_producto, fecha, kilos)

    SELECT cod_producto, fecha, kilos

    FROM inserted;

    INSERT INTO Ventas (cod_producto, fecha, kilos)

    SELECT cod_producto, fecha, kilos

    FROM inserted;

END;
```

Estos ejemplos ilustran cómo funcionan diferentes tipos de triggers y cómo pueden ser utilizados para automatizar acciones o aplicar lógica adicional en respuesta a eventos en la base de datos. Recuerda que los triggers deben ser usados con precaución y tener en cuenta su impacto en el rendimiento y la integridad de los datos.

Cursores

Los cursores en Transact-SQL son estructuras que permiten recorrer y manipular filas de datos en un conjunto de resultados, generalmente obtenidos a través de una consulta SQL. Los cursores son útiles cuando necesitas procesar registros uno por uno, realizar cálculos complejos o manipulaciones en cada fila, o cuando no es posible utilizar otras opciones más eficientes, como conjuntos de resultados y operaciones en conjunto.

Aquí tienes información básica sobre los cursores en Transact-SQL:

Declaración y Uso de un Cursor:

```
USE Verduleria;
```

```
DECLARE @nombre VARCHAR(100);
```

```
DECLARE @precio DECIMAL(10, 2);
```

```
DECLARE miCursor CURSOR FOR
```

```
SELECT nombre, precio
```

```
FROM Productos;
```

```
OPEN miCursor;
```

```
FETCH NEXT FROM miCursor INTO @nombre, @precio;
```

```
WHILE @@FETCH_STATUS = 0
```

```
BEGIN
```

```
    PRINT 'Producto: ' + @nombre + ', Precio: ' + CAST(@precio AS VARCHAR(10));
```

```
FETCH NEXT FROM miCursor INTO @nombre, @precio;  
  
END;  
  
CLOSE miCursor;  
  
DEALLOCATE miCursor;
```

En este ejemplo, hemos declarado un cursor llamado miCursor que recorrerá las filas de la consulta en la tabla Productos. Utilizamos OPEN para abrir el cursor, FETCH NEXT para obtener la siguiente fila y CLOSE para cerrar el cursor cuando hemos terminado.

Tipos de Cursores:

- Cursor FORWARD_ONLY: Se mueve solo hacia adelante a través del conjunto de resultados.
- Cursor STATIC: Crea una copia estática del conjunto de resultados en la memoria.
- Cursor DYNAMIC: Refleja los cambios realizados en los datos subyacentes durante el recorrido.
- Cursor KEYSET: Almacena las claves de las filas en lugar de los datos reales.
- Cursor FAST_FORWARD: Similar a FORWARD_ONLY, pero más eficiente para recorridos en una sola dirección.

Consideraciones Importantes:

- Los cursores pueden ser útiles para escenarios específicos, pero generalmente son menos eficientes que trabajar con conjuntos de resultados.
- Deben usarse con cuidado, ya que pueden causar problemas de rendimiento si no se manejan adecuadamente.

- Es importante cerrar y liberar los cursores después de su uso para liberar recursos.

En general, se recomienda utilizar cursores solo cuando no haya una solución más eficiente. Siempre es una buena práctica buscar alternativas basadas en conjuntos de resultados o técnicas de conjunto cuando sea posible, ya que suelen ser más eficientes y escalables.

Cursor FORWARD_ONLY:

```
USE Verduleria;

DECLARE @nombre VARCHAR(100);

DECLARE miCursor FORWARD_ONLY CURSOR FOR

SELECT nombre

FROM Productos;

OPEN miCursor;

FETCH NEXT FROM miCursor INTO @nombre;

WHILE @@FETCH_STATUS = 0

BEGIN

    PRINT 'Producto: ' + @nombre;

    FETCH NEXT FROM miCursor INTO @nombre;

END;

CLOSE miCursor;

DEALLOCATE miCursor;
```


Cursor STATIC:

```
USE Verduleria;  
  
DECLARE @nombre VARCHAR(100);  
  
DECLARE miCursor STATIC CURSOR FOR  
  
SELECT nombre  
  
FROM Productos;  
  
OPEN miCursor;  
  
FETCH NEXT FROM miCursor INTO @nombre;  
  
WHILE @@FETCH_STATUS = 0  
  
BEGIN  
  
    PRINT 'Producto: ' + @nombre;  
  
    FETCH NEXT FROM miCursor INTO @nombre;  
  
END;  
  
CLOSE miCursor;  
  
DEALLOCATE miCursor;
```

Cursor DYNAMIC:

USE Verduleria;

DECLARE @nombre VARCHAR(100);

DECLARE miCursor DYNAMIC CURSOR FOR

SELECT nombre

FROM Productos;

OPEN miCursor;

FETCH NEXT FROM miCursor INTO @nombre;

WHILE @@FETCH_STATUS = 0

BEGIN

 PRINT 'Producto: ' + @nombre;

 FETCH NEXT FROM miCursor INTO @nombre;

END;

CLOSE miCursor;

DEALLOCATE miCursor;

Cursor KEYSET:

USE Verduleria;

DECLARE @nombre VARCHAR(100);

DECLARE miCursor KEYSET CURSOR FOR

SELECT nombre

FROM Productos;

OPEN miCursor;

FETCH NEXT FROM miCursor INTO @nombre;

WHILE @@FETCH_STATUS = 0

BEGIN

 PRINT 'Producto: ' + @nombre;

 FETCH NEXT FROM miCursor INTO @nombre;

END;

CLOSE miCursor;

DEALLOCATE miCursor;

Cursor FAST_FORWARD:

```
USE Verduleria;

DECLARE @nombre VARCHAR(100);

DECLARE miCursor FAST_FORWARD CURSOR FOR

SELECT nombre

FROM Productos;

OPEN miCursor;

FETCH NEXT FROM miCursor INTO @nombre;

WHILE @@FETCH_STATUS = 0

BEGIN

    PRINT 'Producto: ' + @nombre;

    FETCH NEXT FROM miCursor INTO @nombre;

END;

CLOSE miCursor;

DEALLOCATE miCursor;
```

Cada ejemplo ilustra cómo declarar, abrir, recorrer y cerrar un cursor de un tipo específico en Transact-SQL. Recuerda que los cursores deben usarse con precaución y solo cuando no haya alternativas más eficientes, como trabajar con conjuntos de resultados.

Bibliografía y Webgrafía

Jacobs, P. (2019). SQL: Guía completa para principiantes de la programación SQL con ejercicios y estudios de casos (Libro En Español/SQL Spanish Book Version) (Spanish Edition). Independently Published.

Moestl Vasilik, S. (2016). Problemas prácticos de SQL: 57 desafíos iniciales, intermedios y avanzados para que los resuelvas utilizando un enfoque de «aprender haciendo». Packt Publishing.

Microsoft. (2023, 20 de enero). Introducción a T-SQL. [Curso en línea]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/modules/introduction-to-transact-sql/>

Microsoft. (2023, 20 de enero). Introducción a las consultas con T-SQL. [Ruta de formación]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/paths/get-started-querying-with-transact-sql/>

Microsoft. (2023, 20 de enero). Introducción a la programación T-SQL. [Curso en línea]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/modules/get-started-transact-sql-programming/>

ALTER: Es una instrucción SQL utilizada para modificar la estructura de una tabla existente en una base de datos, como agregar, modificar o eliminar columnas.

CREATE: Es una instrucción SQL que se emplea para crear objetos de bases de datos, como tablas, vistas o procedimientos almacenados.

Cursore: En el contexto de bases de datos, es un mecanismo que permite recorrer y manipular conjuntos de registros en una tabla.

DROP: Es una instrucción SQL que se utiliza para eliminar objetos de bases de datos, como tablas o vistas.

Estructuras de Control: Son instrucciones y técnicas utilizadas para dirigir el flujo de ejecución en programas y procedimientos almacenados, como IF, ELSE, WHILE, entre otras.

Funciones Personalizadas de Usuario: Son funciones creadas por el usuario para realizar operaciones específicas en una base de datos.

GUIDs: Son identificadores únicos globales generados de manera aleatoria, utilizados para garantizar la unicidad de registros en bases de datos distribuidas.

IDENTITY: Es una propiedad que se utiliza en SQL Server para generar automáticamente valores numéricos únicos al insertar registros en una columna.

Índices: Son estructuras de datos utilizadas para acelerar la búsqueda y recuperación de registros en una base de datos, mejorando el rendimiento de las consultas.

STORED PROCEDURE: Es un conjunto de instrucciones SQL almacenadas en la base de datos, que pueden ser ejecutadas en cualquier momento mediante su nombre.

Tipos de parámetros: En el contexto de SQL, se refiere a los diferentes tipos de datos que se pueden utilizar como argumentos en procedimientos almacenados o funciones.

Triggers: Son objetos en una base de datos que se activan automáticamente cuando ocurren ciertos eventos, como una inserción, actualización o eliminación de registros.

TrUNCATE: Es una instrucción SQL utilizada para eliminar todos los registros de una tabla, pero mantiene su estructura y definiciones.

USE: Es una instrucción SQL utilizada para seleccionar una base de datos específica con la que se desea trabajar en un servidor de bases de datos.